ДОМАШНЕЕ ЗАДАНИЕ

Выполнил: студент группы 23Ис-41
Шипунов Даниил Станиславович

1. Выбор веб-приложения и определение используемых форматов

Выбранный сервис: Нейросетевой чат-бот и сервисный помощник Gemini (веб-версия и мобильное приложение от Google).

В рассматриваемом сервисе применяются следующие форматы передаваемых сообщений:

• JSON (over HTTP/HTTPS / SSE): Используется для взаимодействия клиентской части с серверным API по протоколу HTTP/HTTPS с применением технологии Server-Sent Events (SSE). Применяется для отправки пользовательских запросов (промптов), передачи параметров генерации и получения потокового текстового ответа.

• Protocol Buffers / gRPC: Применяется во внутреннем контуре распределенной архитектуры Google для низкоуровневого обмена данными между API Gateway, микросервисами оркестрации и вычислительными кластерами (GPU/TPU).

2. Обоснование выбора формата и анализ альтернатив

Причины использования JSON для клиентского API:

• Нативная поддержка веб-платформами: Фронтенд веб-версии и мобильные клиенты нативно обрабатывают структуры JSON без необходимости внедрения специализированных бинарных парсеров.

• Поддержка потоковой передачи (Streaming): Ответы языковой модели формируются пофрагментно (токенами). Формат JSON позволяет удобно передавать текстовые чанки через SSE и сразу выводить их на экран.

• Простота отладки: Упрощает интеграцию сторонних сервисов с Gemini API и позволяет разработчикам отлаживать запросы через стандартные инструменты DevTools.

Анализ альтернативных форматов:

• XML: Избыточен из-за обязательных закрывающих тегов. В бенчмарках показывает время маршалинга 1355 мкс против 523 мкс у JSON, а объем сообщения достигает 132 КБ (против 113 КБ у JSON). Использование XML при потоковой генерации привело бы к существенному завышению сетевого трафика.

• YAML: Характеризуется низким уровнем скорости парсинга (5318 мкс) и высокой чувствительностью к отступам, что делает его неприменимым для передачи непрерывного потока данных.

• Protobuf (для внешнего API): Обладает высокой скоростью (130 мкс) и компактностью (87 КБ), однако усложнил бы отладку в браузере и потребовал бы дополнительной компиляции кодогенераторов на стороне веб-клиента.

3. Пример данных в трех форматах (Запрос к API Gemini)

Ниже приведен пример структуры запроса на генерацию текста в трех форматах:

Формат А. JSON

```
{
  "model": "gemini-2.5-flash",
  "contents": [
    {
      "role": "user",
      "parts": [
        {"text": "Привет! Объясни простыми словами, что такое квантовые вычисления."}
```

```
      ]
    }
  ],
  "generationConfig": {
    "temperature": 0.7,
    "maxOutputTokens": 300
  }
}
```

Формат Б. XML

```
<?xml version="1.0" encoding="UTF-8"?>
<generateRequest>
    <model>gemini-2.5-flash</model>
    <contents>
        <role>user</role>
        <parts>
            <text>Привет! Объясни простыми словами, что такое квантовые вычисления.</text>
        </parts>
    </contents>
    <generationConfig>
        <temperature>0.7</temperature>
        <maxOutputTokens>300</maxOutputTokens>
    </generationConfig>
</generateRequest>
```

Формат В. Protocol Buffers (Схема .proto)

```
syntax = "proto3";
package gemini.api;
message GenerateRequest {
  string model = 1;
  repeated Content contents = 2;
  GenerationConfig generation_config = 3;
}
message Content {
  string role = 1;
  repeated Part parts = 2;
}
message Part {
  string text = 1;
}
message GenerationConfig {
  float temperature = 1;
  int32 max_output_tokens = 2;
}
```

Сравнительный анализ форматов:

Формат	Читаемость	Компактность	Скорость
JSON	Высокая	Средняя (~113 КБ)	Средняя (523 мкс)
XML	Низкая	Низкая (~132 КБ)	Медленная (1355 мкс)
Protobuf	Нечитаем	Высокая (~87 КБ)	Высокая (130 мкс)

4. Плюсы и ограничения перехода клиентского API с JSON на Protobuf

Преимущества перехода:

• Снижение задержки: Ускорение процесса сериализации и десериализации данных в 3–4 раза, что приводит к сокращению метрики времени до первого токена (TTFT).

• Экономия трафика: Бинарный формат уменьшает объем передаваемых пакетов на 20–30%, что снижает нагрузку на сеть при высокой интенсивности запросов.

• Эволюция схем: Обеспечивает строгое соблюдение типов данных и возможность добавления новых полей без нарушения работы старых версий клиентов.

Возникающие ограничения:

• Усложнение отладки: Потеря человекочитаемости передаваемых пакетов, что затрудняет анализ сетевого трафика в браузере.

• Дополнительные зависимости: Необходимость подключения библиотек парсинга (например, gRPC-Web или Protobuf.js) на стороне клиентских веб-приложений.

5. Пример системы, для которой XML остаётся оптимальным выбором

Оптимальная область: Система межведомственного электронного взаимодействия (СМЭВ), государственные платформы (ФНС, Ространснадзор) и банковские системы на базе SOAP.

Обоснование:

• Поддержка ЭЦП: Стандарт XMLDSig позволяет интегрировать электронно-цифровую подпись (ЭЦП) прямо в структуру документа, обеспечивая юридическую значимость и защиту от изменений.

• Строгая XSD-валидация: Возможность формальной проверки структуры документа на этапе приема с помощью файлов XSD-схем.

• Требования регуляторов: Полное соответствие законодательным требованиям и регламентам государственных органов.